

Name: Deze Yang Student Number: a1943969

Overview

In this submission, I used OpenMP to investigate synchronization correctness and scheduling performance in a shared-memory N-body solver. I first implemented two safe synchronization designs for the reduced-force algorithm, and I then compared four OpenMP implementations to determine how the force-calculation algorithm and scheduling strategy affected runtime, speedup, efficiency and scalability.

Part 2A Synchronization Explanation

I started from the supplied reduced-force implementation, where each particle pair is calculated once and equal and opposite force contributions are accumulated into a shared force array. Parallelising this calculation creates a race condition because different threads can update the same force entry concurrently. In my critical-section version, I protected the shared force updates with an OpenMP critical region, so only one thread could modify the shared array through that region at a time. In my lock-based version, I created one OpenMP lock per particle and acquired the relevant lock before modifying that particle's force. I protected the two particle contributions separately so that a thread did not hold two particle locks simultaneously. In both versions, I used a main parallel region with default(none), explicitly specified data-sharing attributes, and used omp for for the reset, force-calculation and particle-update loops. I relied on the implicit barriers at the end of these work-sharing loops to preserve the required reset -> force calculation -> particle update ordering.

Part 2A Validation

I validated both implementations with the supplied Autograder across different test cases and thread configurations, including 1, 2, 4 and 8 threads. Both the critical-section and lock-based versions passed all correctness tests. I therefore confirmed that each required pairwise interaction was handled correctly and that concurrent force accumulation did not lose shared updates.

Experimental Environment and Methodology

I ran Part 2B in a GitHub Codespaces Ubuntu 24.04 environment using GCC 13.3.0 with OpenMP support and four logical processors. I used 4000 particles, 100 timesteps and a timestep size of 0.01 for every comparison, and I compiled with NO_OUTPUT so that detailed simulation output was disabled during timing. I compared Basic, Reduced Default, Reduced Forces Cyclic and Reduced All Cyclic at 1, 4, 8, 16 and 32 OpenMP threads. I ran every configuration three times and used the arithmetic mean as the reported execution time. I calculated speedup as $S_p = T_1/T_p$ and efficiency as $E_p = S_p/p$. Because the Codespace had four logical processors, the 8-, 16- and 32-thread tests intentionally oversubscribed the available processors.

Results

Table 1. Mean execution time for each implementation (seconds)

Threads	Basic	Reduced Default	Reduced Forces Cyclic	Reduced All Cyclic
1	7.3633	3.7395	3.7231	3.7299
4	3.7287	2.5653	1.9823	1.9237
8	3.7738	2.0448	1.9390	1.9812
16	3.7929	1.9853	1.9638	1.9573
32	3.7649	1.9936	1.9753	2.0295

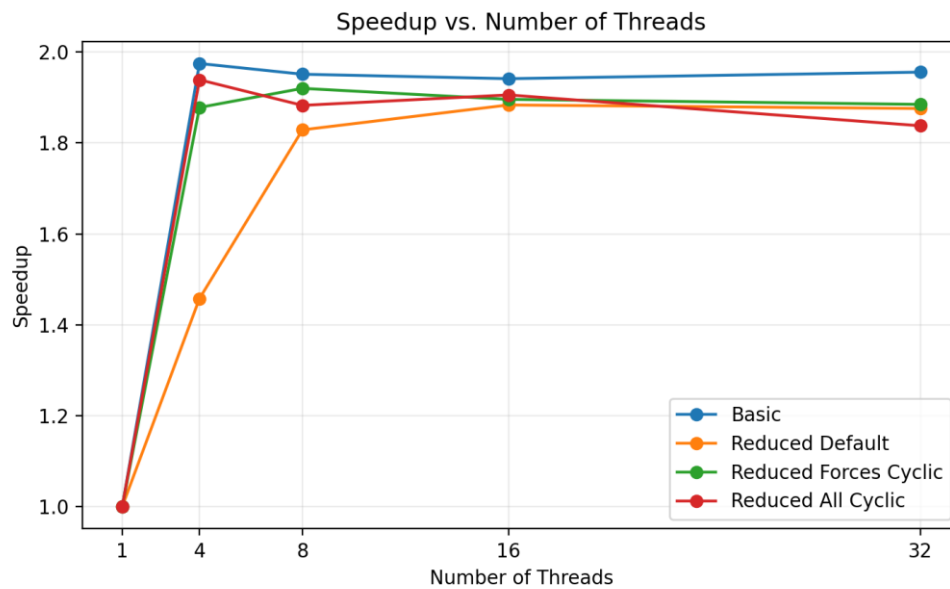


Figure 1. Speedup vs. Number of Threads

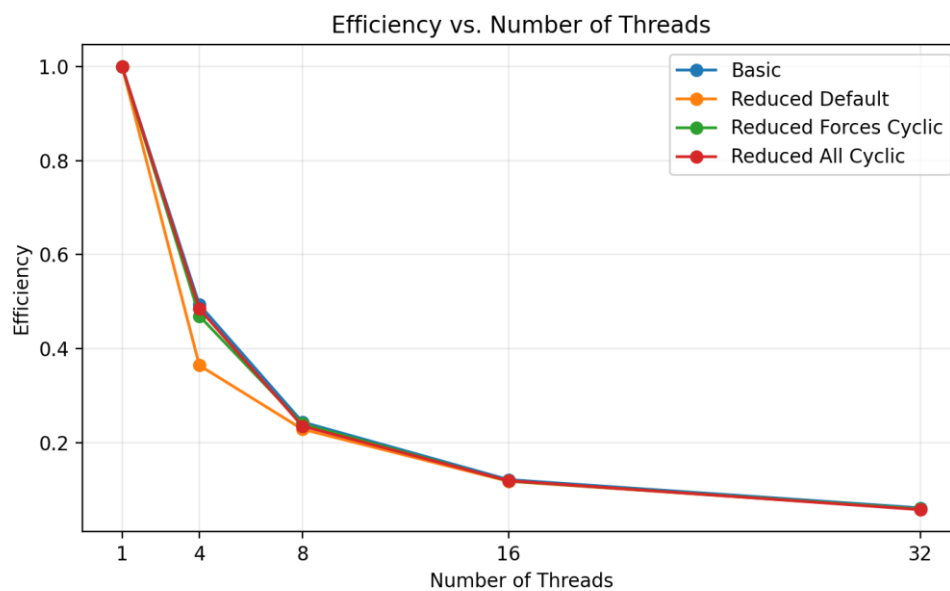


Figure 2. Efficiency vs. Number of Threads

Performance Analysis

My results show that the N-body algorithm had the largest effect on absolute runtime. With one thread, Basic required 7.3633 s, while all three Reduced versions required about 3.72-3.74 s because the reduced algorithm avoids calculating the same particle pair twice. Scheduling also affected performance. At four threads, Reduced Default required 2.5653 s, Reduced Forces Cyclic required 1.9823 s, and Reduced All Cyclic achieved the lowest measured mean runtime of 1.9237 s. I interpret this as a workload-distribution effect: lower-index iterations in the reduced force loop contain more remaining pair interactions, so cyclic scheduling distributes heavy and light iterations more evenly. However, I did not observe a universal benefit from cyclic scheduling. At 32 threads, Reduced All Cyclic increased to 2.0295 s, compared with 1.9753 s for Reduced Forces Cyclic and 1.9936 s for Reduced Default, showing that cyclic scheduling can add overhead when applied to loops that do not need additional load balancing. Scalability was also limited by the four-logical-processor test system. Basic achieved 1.975x speedup at four threads, while Reduced Forces Cyclic and Reduced All Cyclic achieved 1.878x and 1.939x. Increasing the requested thread count beyond four produced little further improvement because 8, 16 and 32 OpenMP threads oversubscribed the available processors. For Reduced Forces Cyclic, efficiency fell from 0.470 at four threads to 0.240 at eight, 0.118 at sixteen and 0.059 at thirty-two, reflecting the lack of additional hardware execution resources and the increasing scheduling and synchronization overhead.

Conclusion

I found that the Reduced algorithm provided substantially better absolute performance than Basic, while cyclic scheduling was most useful for the uneven force-calculation loop. The best measured runtime on my system was Reduced All Cyclic with four threads at 1.9237 s, but my results did not support the assumption that making every loop cyclic always improves performance. Across the full experiment, I found that the reduced-force design with targeted cyclic scheduling provided a strong balance of workload distribution and runtime, while the four-core testing environment was the main practical limit on further parallel scalability.